

Soit la requête XQuery suivante : <pre>Element racine { attribute lang { "fr" } , attribute lang { "en" } }</pre>	-Y a-t-il des erreurs dans cette requête XQuery ? Si oui, corrigez-les.
Soit le code XML suivant : <pre> <book title="XQuery"> <author> <la>Amann</la> <fi>B.</fi> </author> <publisher>O'Reilly</publisher> </book> </book> </pre> Soit la requête XQuery suivante : <pre>element { doc("books2.xml")//book[1]/name(@*[1]) } { attribute { doc("books2.xml")//book[1]/name(*[2]) } { doc("books2.xml")//book[1]/*[2] }</pre>	1.Est-il possible de remplacer /name(@*[1]) par /name(@*) dans cette requête XQuery ? Justifiez votre réponse. 2.Est-il possible de remplacer /name(*[2]) par /string(*[2]) dans cette requête XQuery ? Justifiez votre réponse. 3.Expliquez pourquoi /*[2] et /string(*[2]) génèrent le même résultat dans la dernière ligne de code de cette requête XQuery. 4.Dans quel cas cette requête XQuery génère-t-elle une erreur lorsqu'on ajoute un deuxième attribut au premier nœud d'élément « book » dans ce code XML ?
Soit le code XML suivant : <pre><u who="C" n="9">donc <vocal desc="e"/> ben je prends 17h quand même</u></pre> Soit les expressions XPath suivantes : <pre>//u[contains(., 'quand')] et //u[contains(text(), 'quand')]</pre>	-Expliquez le rôle du point . et de la fonction text() dans ces deux expressions XPath lorsqu'elles sont appliquées à ce code XML et donnez ce qu'elles vont retourner.
Soit les expressions XPath suivantes : <pre>/descendant::titleStmt[2] , //titleStmt[2] , (//titleStmt)[2]</pre>	-Expliquez la différence entre ces 3 expressions XPath.
Soit le code XML suivant : <pre><?xml version="1.0" standalone="no"?> <liste> A<eleve nom="E1" note="12"> L3 ACAD </eleve> B<eleve nom="E2" note="18">L3 ISIL</eleve> </liste></pre> Soit le code JAVA/DOM suivant : <pre>public static void main(String [] args) throws Exception{ Document document = DocumentBuilderFactory.newInstance().newDocumentBuilder().parse(args[0]); NodeList e = document.getDocumentElement().getChildNodes(); for (int o = 0;o<e.getLength();o++){ Filtrer_parasites a = new Filtrer_parasites(); if(a.Filtrer(e.item(o))) System.out.println(e.item(o).getFirstChild().getNodeValue()....().replaceAll(".....", " ")); else if (!e.item(o).getNodeValue().endsWith("...")) System.out.println(e.item(o).getNodeValue()....());}} </pre>	1.À quoi sert l'attribut « standalone » dans ce code XML ? 2.Complétez ce code JAVA/DOM. 3.Que va afficher ce code JAVA/DOM appliqué à ce code XML ? 4.Quel est le rôle de la ligne de code JAVA/DOM en gras ? 5.Est-ce que le résultat sera différent si on remplace la suite de méthodes : « getFirstChild().getNodeValue() » par la méthode « getTextContent() ». Justifiez votre réponse. N.B. La méthode "Filtrer" retourne (noeud.getNodeName() != "#text")
Soit le code XML suivant : <pre><!DOCTYPE name SYSTEM "book_examen.dtd"> <name>W.B<family>Wallace</family></name></pre> Soit la ligne DTD suivante : <!ELEMENT name (.....)>	-Complétez cette ligne DTD ?
Soit la requête XQuery suivante : <pre>element racine { attribute lang { "fr" } , attribute lang { "en" } }</pre>	-Pourquoi cette requête génère-t-elle une erreur ?
Q3. Soit la requête XQuery suivante : <pre>for \$b1 in doc("books1.xml")//book,\$b2 in doc("books2.xml")//book where \$b1/@*=\$b2/@year let \$both:=(\$b2/*/*[2],\$b1/*/*[2]) for \$ele at \$i in \$b2/*/*[2] return <a>{\$ele,\$both[\$i+count(\$both) div 2]}</pre>	-Expliquer cette requête XQuery.
Soit le code XML suivant : <pre><arbres><arbre id="6"><nom_commun>Oranger des osages</nom_commun></arbre>.....</arbres></pre>	1.Par quoi peut-on remplacer "@" dans R1 pour récupérer la valeur d'attribut placé sur l'élément « arbre » ?

Soit la requête R1 XQuery suivante : <pre>for \$arbre in doc("arbres.xml")//arbre return element nom{\$arbre/nom_commun/text()," ID",string(\$arbre/@*)}</pre> Soit la requête R2 XQuery suivante : <pre>for \$nom in doc("arbres.xml")//nom_commun return element nom{\$nom/text()," ID",string(\$nom/.....::.....)}</pre>	2. Compléter la R2, permettant d'afficher le nom commun de l'arbre suivi de son ID. 3. Si on ajoute l'attribut id="6" à l'élément racine du code XML, quelle requête va fonctionner ? Justifier votre réponse.
Soit le code XML suivant : <pre><!DOCTYPE enc SYSTEM "enc_examen.dtd"> <enc><personne datenaissance="a 1942-01-08"/></enc></pre> Soit la ligne DTD suivante : <!ATTLIST personne datenaissance ID #REQUIRED>	1. Pourquoi cette ligne DTD génère-t-elle une erreur ? 2. Proposer deux solutions différentes pour corriger cette erreur ?